MosaForge AI

Mini-rapport de cadrage et d’architecture

Vision. MosaForge AI est un système d’intelligence personnelle **local-first, distribué et agnostique des modèles**. Il combine plusieurs modèles, agents, outils, mémoire et nœuds de calcul afin de fournir une seule expérience d’assistant, sans rendre Claude, OpenAI ou un autre fournisseur indispensable.

MosaForge AI

Intelligence personnelle locale et distribuée

Plusieurs modèles. Plusieurs agents. Une seule intélligence. Votre infrastructure.

Local-first | Modulaire | Agnostique de modèle
Privé | Extensible | Distribué

Problèmes actuels

Limites des solutions existantes

Dépendance au cloud
Nécessite une connexion Internet et dépend de fournisseurs externes.

Coûts et abonnements
Frais récurrents, tokens coûteux et limites d'usage.

Un seul modèle pour tout
Un même modèle est utilisé pour toutes les tâches, même simples.

Confidentialité et données
Les données peuvent être envoyées à des services externes.

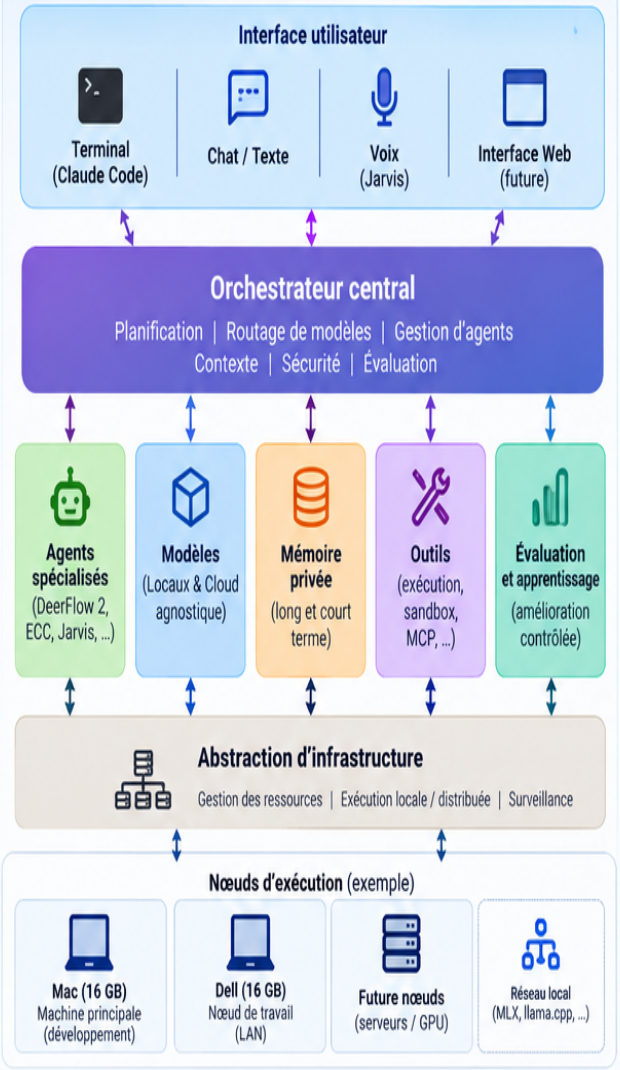
Limites d'une seule machine
Ressources CPU/GPU limitées, difficile pour les gros modèles.

Verrouillage fournisseur
Dépendance à un écosystème (propriétaire ou spécifique).

Gaspillage de contexte/ calcul
Contexte trop large, tâches mal décomposées, utilisation inefficace des ressources.

MosaForge AI

Une architecture unifiée et modulaire



Added Value

Ce que MosaForge apporte

Routage intelligent par tâche
Utilise le bon modèle et le bon agent pour chaque type de tâche.

Exécution locale et offline
Fonctionne sans Internet, avec des modèles et agents locaux (Ollama, etc.).

Délégation multi-agents
Agents spécialisés pour la recherche, le code, l'analyse, la sécurité, etc.

Calcul distribué LAN
Utilise plusieurs machines (Mac, Dell, futurs nœuds) pour exécuter des modèles plus grands.

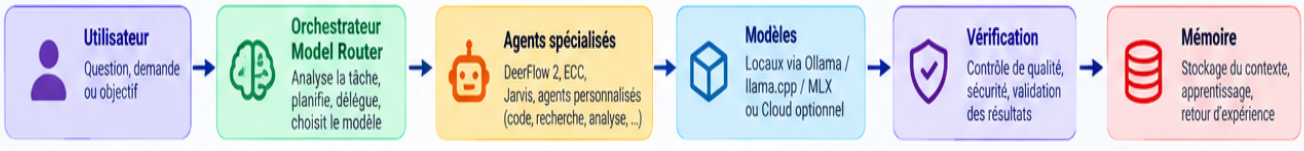
Mémoire privée persistante
Conserve le contexte, les connaissances et les préférences à long terme.

Évaluation et amélioration contrôlée
Apprend des interactions validées, améliore les procédures et le routage.

Cloud optionnel comme fallback
Peut utiliser Claude, OpenAI, etc. pour les tâches difficiles sans en dépendre.

Architecture modulaire
Basée sur des projets existants, facilement extensible et agnostique de modèle.

Flux de fonctionnement (vue simplifiée)



Runtimes de modèles (locaux)

Ollama
Exécution locale simple (API compatible)

llama.cpp
Inférence efficace (+ RPC expérimental)

MLX
Optimisation Apple et calcul distribué (sur plusieurs Mac)

Agents et frameworks (à intégrer)

DeerFlow 2
Super-agent de long terme

ECC
Agents, skills, mémoire, workflows

Jarvis
Assistant personnel direct, local et vocal

Modèles (exemples)

Modèles locaux (exemples)
Qwen | DeepSeek | Kimi | GLM | Autres

Cloud optionnel (fallback)
Claude | OpenAI
Utilisé uniquement si nécessaire

1. Problématique

Les solutions actuelles excellent souvent sur une dimension particulière : agent de développement, recherche autonome, assistant vocal, runtime local ou modèle frontier. Le problème est moins l'absence de briques performantes que leur fragmentation. Un utilisateur peut se retrouver dépendant d'un fournisseur, utiliser un modèle puissant pour des tâches triviales, ou devoir maintenir plusieurs orchestrateurs qui se chevauchent.

Problèmes ciblés : dépendance au cloud, coût et quotas, verrouillage fournisseur, confidentialité, gaspillage de contexte/calcul, limites d'une seule machine et difficulté à coordonner plusieurs modèles spécialisés.

2. Étude de l'existant

Solution	Apport principal	Limite pertinente pour MosaForge
Claude Code	Agent de développement, subagents, plugins, MCP, hooks	Très puissant comme superviseur, mais l'écosystème Claude ne doit pas devenir une dépendance structurelle.
DeerFlow 2	Super-agent, subagents, mémoire, skills, sandbox	Très proche de l'orchestration visée ; risque de chevauchement avec un orchestrateur MosaForge.
ECC	Skills, agents et workflows d'ingénierie	Excellente source de patterns ; à réutiliser sans dupliquer toute la couche d'orchestration.
Jarvis	Assistant local, voix, mémoire, outils	Très pertinent pour l'interface personnelle ; couverture fonctionnelle plus ciblée.
Ollama	Exécution et API de modèles locaux	Runtime, pas un système complet d'autonomie.
llama.cpp	Inférence locale et expérimentation distribuée	Brique d'exécution, avec contraintes de réseau/sécurité pour le RPC.
MLX	Calcul et inférence distribués sur Apple Silicon	Excellent pour un futur cluster Mac, moins universel pour un parc hétérogène.
Open models	Qwen, DeepSeek, Kimi, GLM, etc.	Aucun modèle n'est optimal pour toutes les tâches : besoin de routage et d'évaluation.

3. Valeur ajoutée de MosaForge

Routage intelligent	Choisir le modèle et l'agent selon la difficulté, le coût, la latence et les performances mesurées.
Délégation multi-agents	Faire planifier et vérifier les tâches complexes tout en déléguant les sous-tâches aux agents spécialisés.
Local-first	Continuer à fonctionner sans fournisseur cloud, avec Ollama, llama.cpp, MLX ou d'autres runtimes locaux.
Calcul LAN	Transformer le Mac, le Dell et de futurs nœuds en ressources complémentaires sans alourdir la machine principale.
Mémoire privée	Conserver contexte, connaissances, préférences et procédures sous le contrôle de l'utilisateur.
Amélioration contrôlée	Apprendre d'expériences évaluées : améliorer skills et routage avant de toucher aux poids du modèle.
Cloud optionnel	Utiliser Claude/OpenAI uniquement lorsque le gain attendu justifie le coût ou lorsque les modèles locaux échouent.

4. Architecture cible

Interface → **Orchestrateur** → **Routeur** → **Agents** → **Modèles/Runtimes** → **Vérification** → **Mémoire.** Le Mac reste la station principale de développement. Le Dell devient un nœud LAN. Des serveurs ou GPU peuvent être ajoutés ultérieurement. Les modèles cloud restent des accélérateurs facultatifs.

5. Stratégie matérielle

Phase initiale : investir 0 € dans du matériel supplémentaire. Tester Mac 16 Go + Dell 16 Go sur le LAN. Privilégier d'abord l'architecture multi-agent et la délégation de tâches plutôt que le partage d'un même modèle entre machines. Pour le model-parallelism, une interconnexion rapide devient importante. Apple documente aujourd'hui l'inférence et le fine-tuning distribués de MLX sur plusieurs Macs, notamment via RDMA sur Thunderbolt 5. ■cite■turn0search9■

6. Apprentissage et reinforcement learning

Le RL ne doit pas être le point de départ. MosaForge doit d’abord collecter des expériences vérifiées, mesurer réussite/échec, coût et temps, puis améliorer le routage et les skills. Le fine-tuning vient ensuite. Le RL pourra devenir une piste expérimentale pour apprendre quelles stratégies de routage produisent les meilleurs résultats.

7. POC recommandé

1) Ollama + un ou deux modèles locaux ; 2) abstraction de modèle ; 3) routeur simple ; 4) mémoire locale ; 5) un agent coding et un agent research ; 6) délégation depuis Claude Code ; 7) worker Dell via LAN ; 8) benchmark coût/latence/qualité ; 9) ajout progressif de DeerFlow/ECC/Jarvis selon les résultats. Ollama expose une API locale et une compatibilité OpenAI, ce qui facilite cette première intégration. ■cite■turn0search2■turn0search7■

8. Risques et limites

Le projet ne doit pas prétendre créer un modèle intrinsèquement meilleur que Claude. La valeur est systémique : sélection du bon modèle, parallélisation, mémoire, outils, vérification et utilisation efficace des ressources. Les modèles locaux peuvent être très compétitifs sur certaines tâches, mais leurs performances dépendent du modèle, de la quantification, du matériel et du domaine.

9. Conclusion

MosaForge AI ne cherche pas à remplacer un modèle frontier par un seul modèle open source. Il cherche à construire une infrastructure où plusieurs intelligences spécialisées travaillent ensemble, où le meilleur modèle n’est utilisé que lorsque nécessaire, et où l’utilisateur conserve la maîtrise de ses données, de son matériel et de son architecture.